Lecture 6: Pandas continued

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- WA2, PA2 released yesterday
- Quiz 2 grades and solutions will be released later today

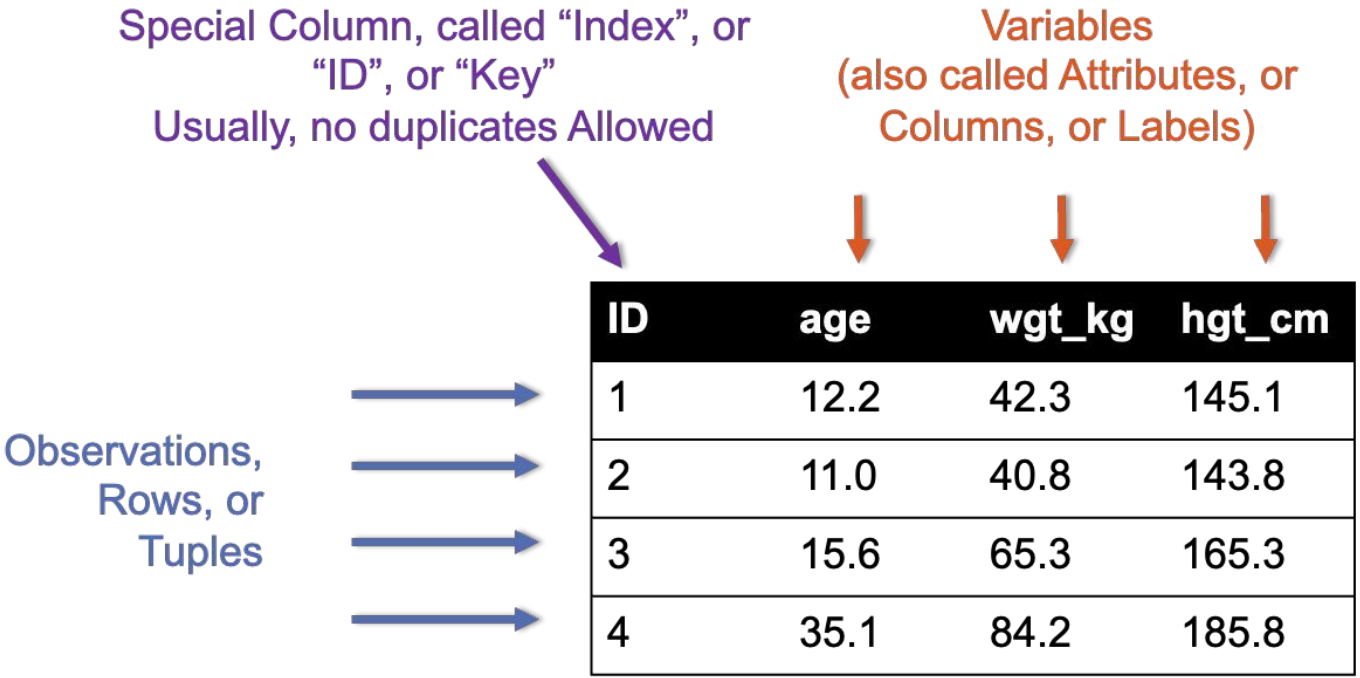
Important tools

- **Python**: A user-friendly language for data analysis.
- **Git**: Helps manage code and data changes when working with a team.
- **Pandas**: Simplifies data cleaning and manipulation.
- **Databases**: Needed for storing and retrieving data efficiently.

Knowing how to use these is essential for effective data analysis, collaboration, and handling data in real-world projects.

Basic Concept Review: Table / Tabular Data

In tabular data, information is organized into rows and columns.



ID	age	wgt_kg	hgt_cm
1	12.2	42.3	145.1
2	11.0	40.8	143.8
3	15.6	65.3	165.3
4	35.1	84.2	185.8

Tabular data is crucial for both Pandas and SQL

Provides a structured and organized way to represent, manipulate and visualize data.

- Both pandas and SQL are commonly used for working with tabular data, making it essential for tasks such as data analysis, manipulation, and visualization.

There are some operations common in both SQL and Pandas

select, join, aggregates etc.

1. SELECT/SLICING/FILTERING

Select only some of the rows, or some of the columns, or a combination

ID	age	wgt_kg	hgt_cm
1	12.2	42.3	145.1
2	11.0	40.8	143.8
3	15.6	65.3	165.3
4	35.1	84.2	185.8

Only columns ID and Age

ID	age
1	12.2
2	11.0
3	15.6
4	35.1

Only rows with wgt_kg > 41

ID	age	wgt_kg	hgt_cm
1	12.2	42.3	145.1
3	15.6	65.3	165.3
4	35.1	84.2	185.8

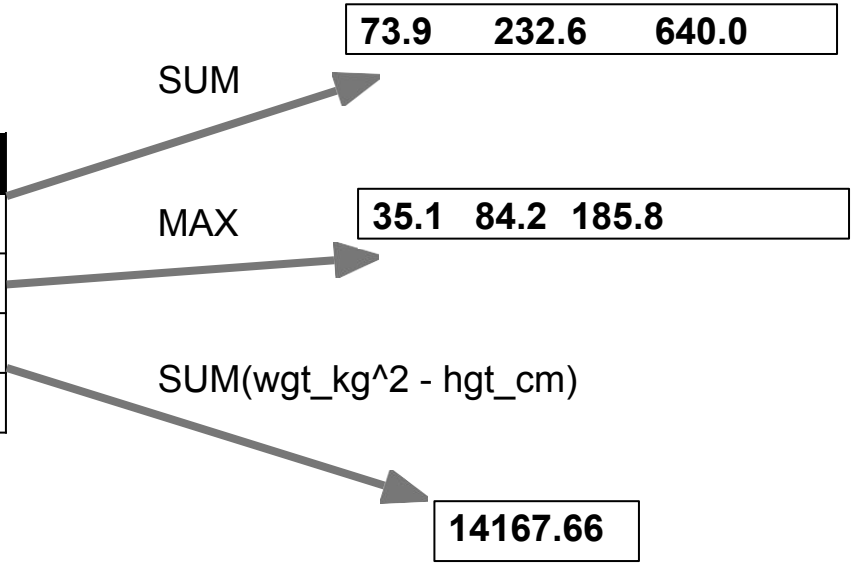
An intersection of the two tables above and to the left

ID	age
1	12.2
3	15.6
4	35.1

2. AGGREGATE/REDUCE

Combine values across a column into a single value

ID	age	wgt_kg	hgt_cm
1	12.2	42.3	145.1
2	11.0	40.8	143.8
3	15.6	65.3	165.3
4	35.1	84.2	185.8



What about ID/Index column?

- Usually not meaningful to aggregate across it
- May need to explicitly add an ID column

3. MAP

Apply a function to every row, possibly creating more or fewer columns

ID	Address
1	College Park, MD, 20742
2	Washington, DC, 20001
3	Silver Spring, MD, 20901

ID	City	State	Zipcode
1	College Park	MD	20742
2	Washington	DC	20001
3	Silver Spring	MD	20901

4. GROUP BY

Group tuples together by column/dimension

ID	A	B	C
1	foo	3	6.6
2	bar	2	4.7
3	foo	4	3.1
4	foo	3	8.0
5	bar	1	1.2
6	bar	2	2.5
7	foo	4	2.3
8	foo	3	8.0

A = “foo”

ID	B	C
1	3	6.6
3	4	3.1
4	3	8.0
7	4	2.3
8	3	8.0

A = “bar”

ID	B	C
2	2	4.7
5	1	1.2
6	2	2.5

4. GROUP BY

Group tuples together by column/dimension

ID	A	B	C
1	foo	3	6.6
2	bar	2	4.7
3	foo	4	3.1
4	foo	3	8.0
5	bar	1	1.2
6	bar	2	2.5
7	foo	4	2.3
8	foo	3	8.0

B = 1

ID	A	C
5	bar	1.2

B = 2

ID	A	C
2	bar	4.7
6	bar	2.5

B = 3

ID	A	C
1	foo	6.6
4	foo	8.0
8	foo	8.0

B = 4

ID	A	C
3	foo	3.1
7	foo	2.3

4. GROUP BY

Group tuples together by column/dimension

ID	A	B	C
1	foo	3	6.6
2	bar	2	4.7
3	foo	4	3.1
4	foo	3	8.0
5	bar	1	1.2
6	bar	2	2.5
7	foo	4	2.3
8	foo	3	8.0

By ‘A’, ‘B’

A = “bar”, B = 1

ID	C
5	1.2

A = “bar”, B = 2

ID	C
2	4.7
6	2.5

A = “foo”, B = 3

ID	C
1	6.6
4	8.0
8	8.0

A = “foo”, B = 4

ID	C
3	3.1
7	2.3

5. GROUP BY AGGREGATE

Compute one aggregate per group

ID	A	B	C
1	foo	3	6.6
2	bar	2	4.7
3	foo	4	3.1
4	foo	3	8.0
5	bar	1	1.2
6	bar	2	2.5
7	foo	4	2.3
8	foo	3	8.0

Group by ‘B’

Sum on C

B = 1

ID	A	C
5	bar	1.2

B = 1

Sum (C)
1.2

B = 2

ID	A	C
2	bar	4.7
6	bar	2.5

B = 2

Sum (C)
7.2

B = 3

ID	A	C
1	foo	6.6
4	foo	8.0
8	foo	8.0

B = 3

Sum (C)
22.6

B = 4

ID	A	C
3	foo	3.1
7	foo	2.3

B = 4

Sum (C)
5.4

5. GROUP BY AGGREGATE

Final result usually seen as a table

ID	A	B	C
1	foo	3	6.6
2	bar	2	4.7
3	foo	4	3.1
4	foo	3	8.0
5	bar	1	1.2
6	bar	2	2.5
7	foo	4	2.3
8	foo	3	8.0

B = 1

Sum (C)
1.2

B = 2

Sum (C)
7.2

B = 3

Sum (C)
22.6

B = 4

Sum (C)
5.4

B	SUM(C)
1	1.2
2	7.2
3	22.6
4	5.4

Pandas

Filtering Data & Applying Statistical Functions

We can filter rows in a DataFrame based on a condition

df[boolean condition]

df[df[‘column’] > x]

Filtering Data & Applying Statistical Functions

We can filter rows in a DataFrame based on a condition and then apply statistical functions to a specific column:

df[boolean condition][column_name].statistics_function()

- **Filters the DataFrame** based on the condition
- It **selects only the rows** that satisfy the condition.

- Selects the **specific column** from the filtered DataFrame obtained in left, resulting in a Series containing only the values from that column.

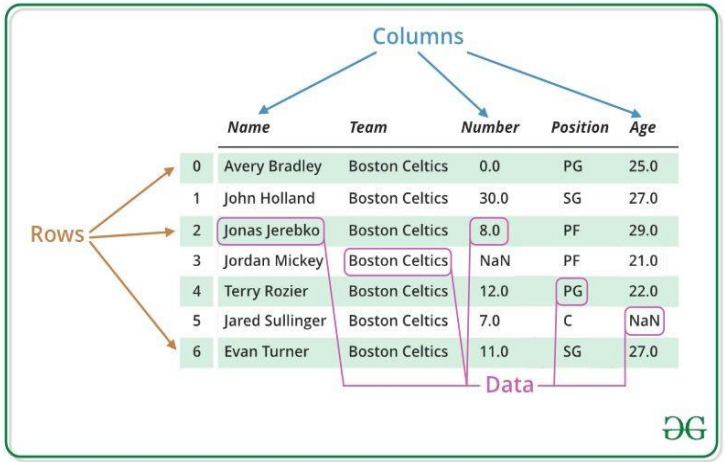
- **Applies** a statistical function to the Series obtained in the left to compute a summary statistic.

Filtering Data & Applying Statistical Functions

We can filter rows in a DataFrame based on a condition and then apply statistical functions to a specific column:

df[boolean condition][column_name].statistics_function()

Try: Calculate the mean of the 'Number' column where the 'Age' is greater than 25



Grouping Data (Groupby)

Purpose: The groupby() operation splits data into groups based on a column/criteria, and then apply a function (like sum, mean, etc.) to each group, and combines results.

- This process is commonly referred to as "split-apply-combine."

```
df.groupby('condition')[target_column_name].statistics_function()
```

grouping_column

Splitting

- Groups the DataFrame based on the values in the 'condition' column.
- **Splits the DataFrame** into groups based on unique (identical) values in the specified column.

Select

- Selects a specific column from each

Apply

- **Applies** a function to each group of values

Common Usage:

- **# Group by a single column**

```
grouped = df.groupby('column_name')
```

- **# Group by multiple columns**

```
grouped = df.groupby(['col1', 'col2'])
```

df.groupby('condition')[target_column_name].statistics_function()

grouping_column

Example:

- **# Group by and apply aggregation**

```
df.groupby('category')['value'].mean()
```

- `df.groupby('condition')` - Groups the DataFrame by the column `'condition'`
- `[column_name]` - Selects a specific column (`'value'` in your example) from each group. [OPTIONAL]
- `.statistics_function()` - Applies an aggregation function (like `mean()`, `sum()`, `count()`, etc.)